



PDF Download
3712285.3759819.pdf
23 December 2025
Total Citations: 0
Total Downloads: 2834



Published: 16 November 2025

Citation in BibTeX format

SC '25: The International Conference
for High Performance Computing,
Networking, Storage and Analysis
November 16 - 21, 2025
MO, St. Louis, USA

Conference Sponsors:
SIGHPC

Latest updates: <https://dl.acm.org/doi/10.1145/3712285.3759819>

RESEARCH-ARTICLE

Microscopic-Level Mouse Whole Cortex Simulation Composed of 9 Million Biophysical Neurons and 26 Billion Synapses on the Supercomputer Fugaku

RIN KURIYAMA, The University of Electro-Communications, Chofu, Tokyo, Japan

KAAYA AKIRA, The University of Electro-Communications, Chofu, Tokyo, Japan

LAURA GREEN

BEATRIZ HERRERA

KAEL DAI

MARI IURA, The University of Electro-Communications, Chofu, Tokyo, Japan

[View all](#)

Open Access Support provided by:

The University of Electro-Communications

Yamaguchi University

RIKEN Center for Computational Science

Research Organization for Information Science and Technology

Microscopic-Level Mouse Whole Cortex Simulation Composed of 9 Million Biophysical Neurons and 26 Billion Synapses on the Supercomputer Fugaku

Rin Kuriyama*
The University of
Electro-Communications
Tokyo, Japan
rin.kuriyama@uec.ac.jp

Kaaya Akira*
The University of
Electro-Communications
Tokyo, Japan
kaaya4502@mail.uec.jp

Laura Green
Allen Institute
Seattle, USA
laura.green@alleninstitute.org

Beatriz Herrera
Allen Institute
Seattle, USA
beatriz.herrera@alleninstitute.org

Kael Dai
Allen Institute
Seattle, USA
kaeld@alleninstitute.org

Mari Iura
The University of
Electro-Communications
Tokyo, Japan
mari.iura@uec.ac.jp

Gilles Gouaillardet
The Research Organization for
Information Science and Technology
Hyogo, Japan
gilles@rist.or.jp

Asako Terasawa
The Research Organization for
Information Science and Technology
Hyogo, Japan
a.terasawa@rist.or.jp

Taira Kobayashi
Yamaguchi University
Yamaguchi, Japan
tairakobayashi@yamaguchi-u.ac.jp

Jun Igarashi
RIKEN Center for Computational
Science (R-CCS)
Hyogo, Japan
jigarashi@riken.jp

Anton Arkhipov
Allen Institute
Seattle, USA
antona@alleninstitute.org

Tadashi Yamazaki
The University of
Electro-Communications
Tokyo, Japan
yamazaki.tadashi@uec.ac.jp

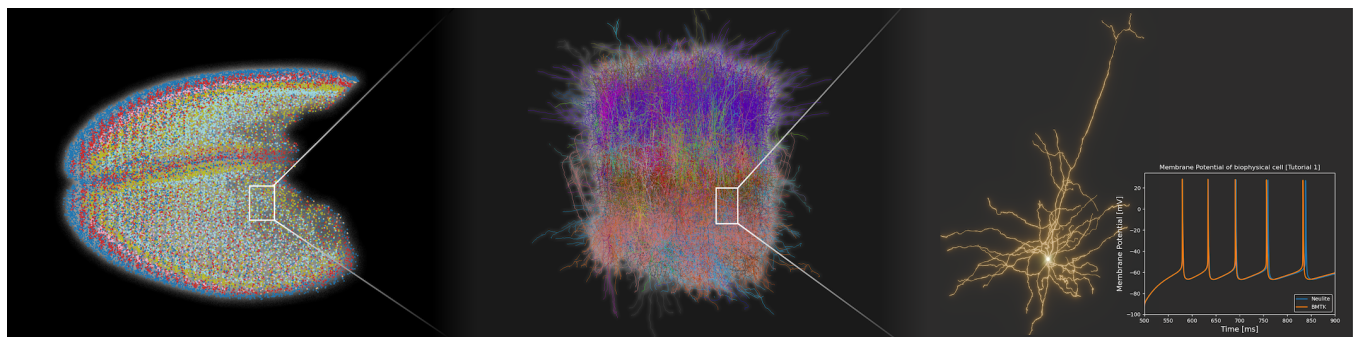


Figure 1: A mouse whole cortex model simulation on the Supercomputer Fugaku. Left: A whole cortex is reconstructed by arranging biophysical neurons and synaptic connections in a 3D space. Colors represent different cell types. Middle: A local circuit reflects a columnar structure of the cortex composed of 6 layers. Right: Each neuron has its characteristic morphology, ion channels and calcium concentration, thereby exhibiting highly nonlinear dynamics.

*Both authors contributed equally to this research.



This work is licensed under a Creative Commons Attribution 4.0 International License.
SC '25, St Louis, MO, USA
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1466-5/25/11
<https://doi.org/10.1145/3712285.3759819>

Abstract

Understanding the brain remains a major scientific challenge due to its complex structure and function. Unlike artificial neural networks, the biological brain features diverse biophysical properties essential to its function. Building an accurate digital replica using extensive anatomical and physiological data, which are available in standardized public databases, has emerged as a promising approach. In this

study, a light-weight biophysical neuron simulator was developed and optimized to the supercomputer Fugaku. Good strong scaling was demonstrated in a benchmark model upto 152,064 compute nodes with 7.13 petaflops performance. In a more realistic scenario, a whole cerebral cortex of a mouse, consisting of 9 million biophysical neurons and 26 billion synapses, was simulated on the full-scale Fugaku with 145,728 nodes. These suggest that the present high-performance computing technology is ready to support the construction of a digital replica of the whole mammalian brain.

CCS Concepts

• **Applied computing** → **Systems biology**; *Computational biology*; • **Computing methodologies** → *Multiscale systems*.

Keywords

Brain, Neuron, Synapse, Biophysical, Simulation, Fugaku

ACM Reference Format:

Rin Kuriyama, Kaaya Akira, Laura Green, Beatriz Herrera, Kael Dai, Mari Iura, Gilles Gouaillardet, Asako Terasawa, Taira Kobayashi, Jun Igarashi, Anton Arkhipov, and Tadashi Yamazaki. 2025. Microscopic-Level Mouse Whole Cortex Simulation Composed of 9 Million Biophysical Neurons and 26 Billion Synapses on the Supercomputer Fugaku. In *The International Conference for High Performance Computing, Networking, Storage and Analysis (SC '25)*, November 16–21, 2025, St Louis, MO, USA. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3712285.3759819>

1 Introduction

Understanding the neural mechanisms of the human brain is one of the grand challenges in science [24]. Although we are able to mimic human behaviors through generative artificial intelligence (AI), the internal mechanisms of AI tools are likely dissimilar to that of the human brain, because the biological brain is far more complex than deep networks. Biological neurons exhibit various characteristic morphologies, are equipped with a number of non-linear properties related to ion channels and intracellular calcium concentration, and transmit information in an all-or-none manner by emitting electrical pulses called spikes. These are contrastive to the rectified linear unit (ReLU) used in deep neural networks. Furthermore, connections between neurons via junctions called synapses are nonuniform, recurrent, and highly sparsified in the biological brain. It is possible that those fine structural and functional properties are irrelevant to the human brain function, but many lines of experimental evidence point to the importance of such properties [27]. Therefore, instead of abstracting the formation of the brain, researchers are now creating digital replicas of the brain. This work was pioneered by the Blue Brain Project [29] and followed by several national projects such as Human Brain Project and EBRAINS in the EU [2], Brain Initiative in the US [34], and Brain/MINDS 1.0 and 2.0 in Japan [32].

To create a digital replica of the brain, huge amounts of anatomical and physiological data are necessary. Recent advances in recording techniques allow us to obtain such data. For example, volume electron microscopy allows us to reconstruct the anatomical structure of the mammalian brain at 1 mm³ in nanometer scales [10, 39, 43]. Neuropixels record electrical activity of hundreds of neurons simultaneously [41]. Moreover, such raw data are

manipulated into a standardized data format, and stored in public databases. The Allen Institute for Brain Science is a leading organization that maintains various databases from genes to functions. In particular, the Allen Cell-Types Database (CTDB) [22, 42] is a huge collection of individual neuron data that includes both morphology and electrical properties in standardized data formats [12, 36].

Based on the available open data, attempts have been made to reconstruct local circuits of the brain at a fine spatiotemporal resolution [5, 13, 30]. Computer simulations of those circuits should be able to reproduce and even predict the neural activity observed experimentally, which will help us to advance our understanding of how the brain works [3]. However, to achieve this, the efficient use of supercomputers will be essential for large-scale numerical simulations.

In this study, Japan's flagship supercomputer Fugaku was used to build and simulate a digital replica of the mammalian brain. To harness the unprecedented computational power of Fugaku, we developed software for this purpose. It showed good scaling properties on a benchmark model, and we were able to conduct a simulation of a whole cerebral cortex (mouse) using the full-scale nodes of Fugaku.

2 Methods

2.1 Modeling and simulating biophysical neurons and synapses

This subsection briefly describes how to model neurons and synapses mathematically and simulate them numerically so as to give readers a general feeling of brain simulation at a microscopic level.

2.1.1 Mathematical Methods. Biological neurons are highly non-linear elements with distinct properties. A neuron has a characteristic morphology consisting of the soma, dendrites, and an axon (Figure 2). The dynamics of a neuron are described by changes in intracellular voltage (membrane potential, v) and calcium concentration (ca), which can take different values with time (t) and location inside the cell (x).

The passive dynamics of v can be described by the following diffusion equation:

$$\tau \frac{\partial v}{\partial t} = -(v(t, x) - E_{\text{leak}}) + \lambda^2 \frac{\partial^2 v}{\partial x^2}, \quad (1)$$

where τ and λ are the decay time constant and the electrotonic length, respectively. The first and second terms in the right-hand side (RHS) are the decay and diffusion terms, respectively, where E_{leak} is the voltage at the equilibrium called reversal potential. The decay term represents the leakage of the current through the cell membrane, whereas the diffusion term represents the flow of the current inside the cell. By taking the second-order difference onto the diffusion term, we obtain a set of ordinary differential equations (ODEs).

On the other hand, the cell membrane is not passive. Ion channels, which are pores on the membrane surface that open/close actively, generate current flows across the cell membrane depending on the membrane potential and calcium concentration at the location. Thus, Eq. (1) cannot be applied directly to describe the neuron dynamics. Instead, the multicompartment modeling approach has been taken in general. In this approach, the morphology of a

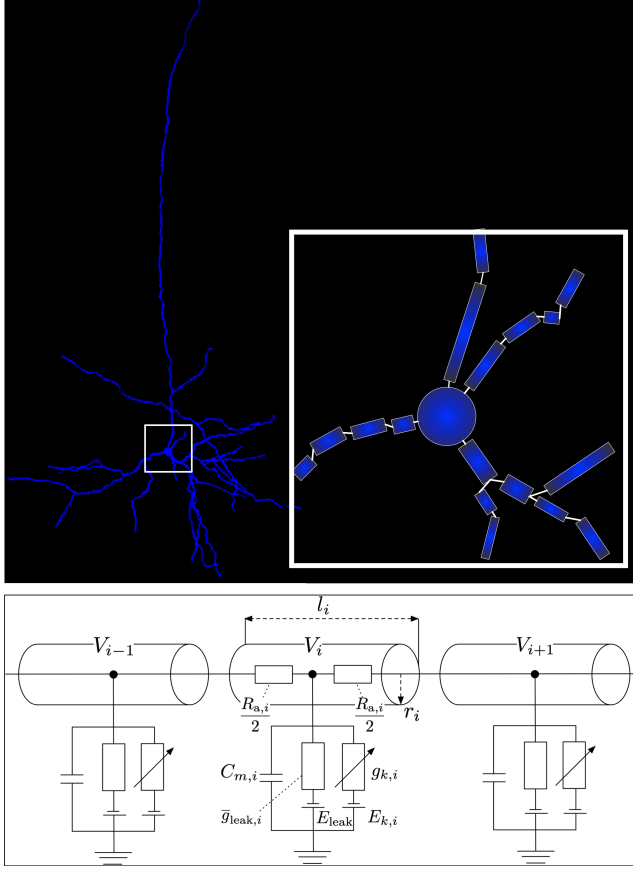


Figure 2: Schematic of compartmentalization of neuron morphology. Top: Morphology of an example neuron, Scnn1a_473845048 in CTDB. Only the soma (center ellipse) and dendrites (tree-like structures) are shown, because the axon was too thin to visualize. The inset shows compartmentalization for the center part inside the white rectangle. The center ball and cylinders represent the soma and segments of dendrites. They are connected by axial resistances shown by a thin white line. Bottom: Example representation of a single dendritic branch as a series of 3 compartments. The middle compartment i is a cylinder of length l_i and radius r_i . It has the membrane potential V_i , a passive mechanism composed of a capacitor with capacitance $C_{m,i}$, a resistor with leak conductance $\bar{g}_{leak,i}$ and a battery with reversal potential $E_{leak,i}$. It also equips active ion channels and synaptic inputs by a set of a variable resistors with conductance $g_{k,i}$ and a battery with reversal potential $E_{k,i}$, respectively (only one set is depicted). The middle compartment is connected to the left and right ones ($i-1$ and $i+1$) with axial resistance $R_{a,i}$.

neuron is split into short equipotential segments called compartments connected by axial resistances, thereby discretised in space ($v(t, x) \rightarrow v_i(t)$), where i represents the compartment ID. Then, the

dynamics of v_i can be written as follows:

$$C_{m,i} \frac{dv}{dt} = -\bar{g}_{leak,i} (v_i(t) - E_{leak,i}) + \sum_j \frac{v_j(t) - v_i(t)}{R_{a,i,j}} + I_{ion,i}(t) + I_{syn,i}(t) + I_{ext,i}(t), \quad (2)$$

where $C_{m,i}$, $\bar{g}_{leak,i}$, $R_{a,i,j}$ are constants representing the capacitance and leak conductance of compartment i , and the axial resistance between compartments i and j . When a one-dimensional cylinder with radius r is discretised by taking the second-order central differences with axial resistance $R_{a,i,j}$, Eqs. (1) and (2) become identical under the passive condition ($I_{ion,i}(t) = I_{syn,i}(t) = I_{ext,i}(t) = 0$), because there is a relationship: $\tau = C_{m,i}/\bar{g}_{leak,i}$ and $\lambda = \sqrt{r_i/2R_{a,i,j}\bar{g}_{leak,i}}$. Now, we have three more active terms in the RHS: a current generated through ion channels on the membrane surface $I_{ion,i}(t)$, that generated by spike inputs via synapses $I_{syn,i}(t)$, and that injected directly to typically control the baseline activity $I_{ext,i}(t)$.

Intracellular current through ion channels $I_{ion,i}(t)$ for compartment i is written as follows:

$$I_{ion,i}(t) = - \sum_{k=1}^K \bar{g}_{k,i} a_{k,i}(v_i, ca_i)^{l_{k,i}} b_{k,i}(v_i, ca_i)^{m_{k,i}} (v_i(t) - E_{k,i}). \quad (3)$$

The RHS represents the sum of the currents generated through K types of ion channels. Each type has its own peak conductance $\bar{g}_{k,i}$ and reversal potential $E_{k,i}$, and is modulated by two gate variables ($a_{k,i}$, $b_{k,i}$) depending on the membrane potential and calcium concentration, and the exponents ($l_{k,i}$, $m_{k,i}$). There are 15 types of ion channels (i.e., $K = 15$) defined for neurons in the CTDB (see below).

Gate variables control open/close (activation/inactivation) of the corresponding channel. Most gate variables are described by a first-order ODE:

$$\tau_x \frac{dx}{dt} = -(x(t) - x_\infty), \quad (4)$$

where $x \in \{a_k, b_k\}$ is a gate variable, τ_x is the time constant and x_∞ is the equilibrium value. Both are functions of $v_i(t)$ and $ca_i(t)$, and defined for each channel. One exception is a fast sodium channel (NaV), which is described by a 15-variable Markov process.

Synaptic current $I_{syn,i}(t)$ for compartment i is modeled as follows:

$$I_{syn,i}(t) = - \sum_{k=1}^K g_{k,i}(t) (v_i(t) - E_{k,i}), \quad (5)$$

where K synapses are assumed to contact compartment i , and $g_{k,i}(t)$ and $E_{k,i}$ respectively represent the synaptic conductance and reversal potential of the k -th synapse. Two types of synapses exist in general: excitatory synapses and inhibitory synapses depending on the presynaptic neurons that provide spike inputs. The reversal potential is set at that of sodium or potassium channels, respectively.

The synaptic conductance $g_{k,i}(t)$ changes based on the input spikes as follows:

$$g_{k,i}(t) = \bar{g}_{k,i} \sum_{f \in F_i} \exp\left(-\frac{t - t^{(f)} - \delta_{k,i}}{\tau_{k,i}}\right) \Theta(t - t^{(f)} - \delta_{k,i}), \quad (6)$$

where $\bar{g}_{k,i}$, $\tau_{k,i}$ and $\delta_{k,i}$ are constants for the peak conductance (synaptic weight), decay time constant and synaptic transmission delay, respectively. F_i represents the spike train to the compartment,

and $t^{(f)}$ is the time that the spike f arrived. $\Theta(t)$ is the Heaviside step function.

The calcium concentration ca_i is calculated by the calcium-mediated current $I_{ca,i}(t)$ defined as

$$I_{ca,i}(t) = -g_{Ca_HVA,i}(t)(v_i(t) - E_{Ca,i}) - g_{Ca_LVA,i}(t)(v_i(t) - E_{Ca,i}), \quad (7)$$

where $g_{Ca_HVA,i}$ and $g_{Ca_LVA,i}$ are conductances of the high- and low-voltage-activated calcium channel, respectively, and $E_{Ca,i}$ is the reversal potential for the calcium-mediated current. Then, ca_i is changed by a first-order ODE as follows:

$$\frac{dca_i}{dt} = -\frac{1}{\tau_{ca}}(ca_i(t) - ca_{\min}) + 10000(I_{ca,i}(t)Y_{ca}/2FD), \quad (8)$$

where τ_{ca} is the decay time constant, $ca_{\min}$ is the minimum concentration, Y_{ca} is the amplified parameter, F is the Faraday constant, and D is a constant specified by the depth of the shell of the calcium buffer.

Morphology and electrical property of neurons are stored in CTDB and publicly available as open data [22]. Morphology data are stored in a standard format called SWC [4]. Electrical property includes parameters for passive and 15 predefined active ion channels (NaTs, NaV, NaTa, NaP, Ih, Ca_HVA, Ca_LVA, SK, Im, IM_v2, Kv3_1, K_T, K_P, Kd, Kv2like). These data are provided in a standardized data format called SONATA [12]. Datasets for synaptic parameters are publicly available as well [8].

2.1.2 Numerical methods. All ODEs are numerically solved for each time step Δt . Δt was set at either 0.1 msec for balanced random network model (Section 2.4.1) or 0.005 msec for a mouse whole cortex model (Section 2.4.2) of virtual time. For each time step, (a) synaptic currents and (b) ionic currents are calculated, and then (c) calcium concentration and (d) membrane potential are calculated in this order for each neuron.

For (a) synaptic currents $I_{syn,i}(t)$ in Eq. (5), first, Eq. (6) was calculated by a subtractive method [40] as follows:

$$g_{k,i}(t + \Delta t) = \bar{g}_{k,i} \text{sum}_{0,i}(t + \Delta t), \\ \text{sum}_{0,i}(t + \Delta t) = (1 - \exp(-\Delta t/\tau_{k,i})) \text{sum}_{0,i}(t) + s_i(t), \quad (9) \\ \text{sum}_{0,i}(0) = 0,$$

where $s_i(t) = 1$ if compartment i of the postsynaptic neuron received a spike, and 0 otherwise. Then, the synaptic current was calculated as in Eq. (5). For (b) ionic currents $I_{ion}(t)$ in Eq. (3), first, all gate variables in Eq. (4) were updated by a first-order integral method as follows:

$$x(t + \Delta t) = x_{\infty} + (x(t) - x_{\infty}) \exp(-\Delta t/\tau_x). \quad (10)$$

Then, the currents were calculated as in Eq. (3). For (c) calcium concentration $ca_i(t)$, Eq. (8) were updated by a standard forward Euler method. Finally, for (d) membrane potential $v_i(t)$ in Eq. (2), we took a central difference with respect to t . That is, first we advanced time by $\Delta t/2$ implicitly, and then further advanced by $\Delta t/2$ explicitly. Because x in Eq. (1) was discretised by the second-order central difference, the resulting method was the same with the Crank-Nicolson method [25]. For the former $\Delta t/2$, we used a backward Euler method, where we had to solve a linear equation system. Here, the morphology of neurons are tree-like structure without loops. Thus, the resulting matrix is symmetric and almost

a tri-diagonal matrix with a few non-diagonal elements at dendritic branches. We can solve the linear equation easily by a modified Gaussian elimination called Hines' method [19], even in parallel [20, 44]. For the latter $\Delta t/2$, a forward Euler method was applied.

After updating the membrane potential, we performed spike detection, where we tested if a neuron emits a spike by checking the membrane potential exceeds a threshold set at -15 mV. A list of IDs of neurons that emit spikes was created for each 1 msec of virtual time. That means, spikes were not propagated instantly whenever the membrane potential exceeded the threshold. This assumption is valid when we take the minimal transmission delay of 1 msec into account [35]. For efficiency, we approximated the calculation of (a) synaptic currents for each 1 ms after the spike propagation.

2.2 Software Development

To model and simulate large and complex biophysical neural networks, we developed a light-weight neuron simulator "Neulite". Neulite consists of two parts: the front-end (modeling interface) and the back-end (simulation kernel).

2.2.1 Modeling Interface. To build biophysical network models easily and efficiently, Neulite uses the Brain Modeling ToolKit (BMTK), developed by the Allen Institute, as its modeling interface [11]. BMTK is a Python package composed of a few modules (a builder module and multiple simulator modules for multiple simulators; Figure 3). In particular, the bionet module is a simulator module that carries out biophysical simulations using NEURON simulator [9] as its back-end system. Bionet reads a model script and generates *.hoc and *.mod files, which are necessary files for NEURON simulator.

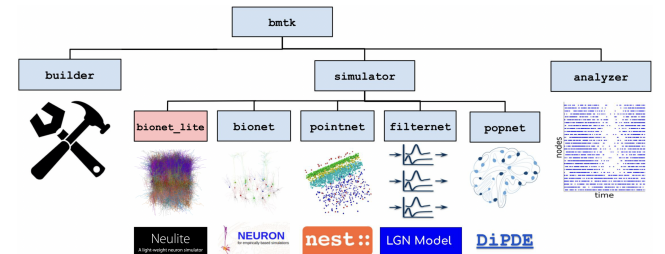


Figure 3: Organization of BMTK and our extension. A simulator module bionet_lite was added in this study (shown in pink). This figure was modified from [11].

We developed the "bionet_lite" module, which is a drop-in replacement of the bionet module, for the simulation kernel of Neulite (Section 2.2.2). Bionet_lite reads the same model script and generates *.csv files for the kernel. With a minimal rewriting (rewriting all bionet words in a BMTK script to bionetlite), the same model should run in Neulite. Thus, bionet and bionet_lite modules assume different simulators (i.e., NEURON simulator and the Neulite kernel, respectively) as their back-end systems. Further discussion on the development will be made in Section 4.1.

2.2.2 Simulation Kernel. The simulation kernel (Neulite kernel) reads the CSV files generated by the bionet_lite module and performs a numerical simulation. The kernel is written in C17

with a clean data structure. Three strategies were taken for the kernel development. First, all necessary preprocessing is done by the front-end system. Second, everything is determined at compile time except what must be put on the heap memory in runtime. Third, the kernel supports only the neuron models in CTDB. These made the reference implementation of the kernel simple; it consists of less than 2,600 lines of code. Furthermore, the simple implementation allowed us to easily extend the kernel, including the optimization to Fugaku (Section 2.3).

2.2.3 Example: Single Cell Simulation with Current Injection. Here, we present how to use Neulite based on a tutorial of BMTK: “Tutorial: Single Cell Simulation with Current Injection (with BioNet)” on the BMTK website ¹.

The basic strategy is to rewrite a script in BMTK. First, users import the `bionet_lite` module at the header of the script. Then, the term “bionet” in the script is replaced with “bionetlite” at each instance (Figure 4) Because the `bionet_lite` module is a drop-in replacement for the `bionet` module, this minimal rewriting is enough.

```
from bmtk.builder import NetworkBuilder
+ from bmtk.builder.network_adaptors.neulite_network import NeuliteNetwork
- net = NetworkBuilder("mcortex")
+ net = NetworkBuilder("mcortex", NeuliteNetwork)
net.add_nodes(
    : (Omitted)
    dynamics_params='472363762_fit.json',
    morphology='Scnn1a_473845048_m.swc'
)
net.build()
net.save_nodes(output_dir='sim_ch01/network')
: (Omitted)
- from bmtk.utils.sim_setup import build_env_bionet
+ from bmtk.utils.sim_setup import build_env_bionetlite
- build_env_bionet(
+ build_env_bionetlite(
    base_dir='sim_ch01', # Where to save the scripts and config files
    config_file='config.json', # Where main config will be saved.
    network_dir='network', # Location of directory containing network files
    tstop=2000.0, dt=0.1, # Run a simulation for 2000 ms at 0.1 ms intervals
    report_vars=['v', 'cal'], # Tells simulator what we want to record
    current_clamp={ # Creates a step current from 500.0 ms to 1500.0 ms
        'amp': 0.120,
        'delay': 500.0,
        'duration': 1000.0
    },
    include_examples=True, # Copies components files for tutorial examples
    compile_mechanisms=True # Will try to compile NEURON mechanisms
)
```

Figure 4: Example usage of the `bionet_lite` module for single cell simulation with current injection. Lines in orange were replaced with those in blue.

Then, users run the rewritten script to generate necessary files: a description of neuron populations (`population.csv`) and that of synaptic connections (`connection.csv`) in CSV format, and parameters for simulation setting in a C header file. After compiling the kernel with the header file, users run the generated executable file with the CSV files as arguments to start a simulation. Finally, users obtain the result of the simulation as the spike data in CSV format, which are a list of tuples (time, neuron ID) that represents the time when the neuron emitted a spike.

2.3 Optimization of the Kernel to Fugaku

The simulation kernel is a general implementation without any code optimization for specific computer architectures. We developed a kernel highly optimized to Fugaku.

Fugaku is Japan’s flagship supercomputer jointly developed by RIKEN and Fujitsu [38] and installed at RIKEN Center for Computational Science in Kobe. It consists of 158,976 compute nodes

¹https://alleninstitute.github.io/bmtk/tutorials/tutorial_01_single_cell_clamped.html

and shows the effective performance of 440 petaflops in double precision. Each compute node equips 32GB HBM2 memory and an ARM-based Fujitsu A64FX processor (Armv8.2-A SVE), which is split into 4 compute management groups (CMGs) with 12 compute cores. Each core has 32 registers and 2 SIMD units that execute Scalable Vector Extension (SVE) operations of 512 bits. Thus, 8 variables in double precision can be computed in parallel. The inter-node communication is made via Tofu interconnect D with 40GB/sec bandwidth.

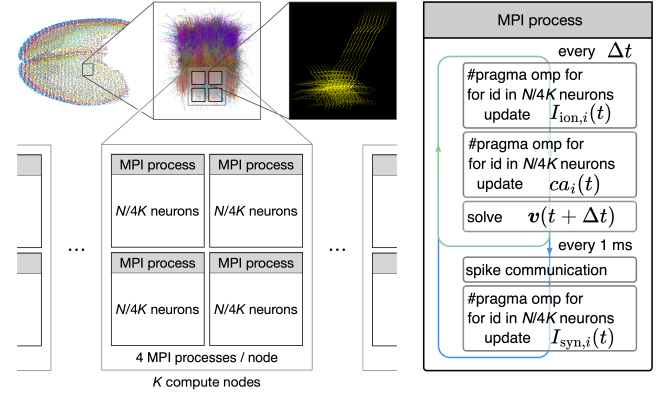


Figure 5: Process- and thread-level parallelization for a large-scale neural network simulation. Left: A target network is divided into subnetworks of $N/4K$ neurons by process-level parallelization. Each subnetwork is handled by one of $4K$ MPI processes. Right: Calculation on 1 MPI process. Neuronal parameters are updated at every time step Δt by thread-level parallelization, while spike communication is performed at every 1 msec.

To carry out a biophysical simulation efficiently while harnessing the hierarchical architecture, the target network model was split hierarchically (Figure 5). Let N be the number of neurons in the target network, and K be the number of compute nodes that can participate. We here used 4 MPI processes and 12 OpenMP threads per node, based on the natural decomposition of Fugaku consisting of 4 CMGs per node and 12 cores per CMG. First, the entire network was split into $4K$ smaller subnetworks to invoke $4K$ MPI processes, where each subnetwork contains about $N/4K$ neurons. Second, for each subnetwork, necessary calculation for the $N/4K$ neurons, which are calculation of (a) synaptic currents, (b) ionic currents and (c) calcium concentration explained in Section 2.1.2, were split into 12 threads and executed in parallel using OpenMP, because 1 CMG contains 12 compute cores. Third, the calculation of (d) membrane potential, specifically, calculation of the linear equation was performed using a parallel version of Hines’ method [20, 44] on 12 threads. In Hines’ method, because the morphology of neurons is a tree-like structure without loops, the coefficient matrix is stored in a compact form consisting of three arrays: a diagonal value array, a parent-child value array, and a parent index array. This structure enables Gaussian elimination to be performed without fill-in during the calculation. A parallel version of Hines’ method [20, 44] splits branches and computes them in parallel. In our implementation for

Fugaku, branches were assigned to 12 threads by equalizing the total size of branches, rather than the number of branches. Furthermore, to achieve SIMD-level parallelism, the full set of equations for 8 morphologically identical neurons was grouped and solved simultaneously using a SIMD unit. This SIMD implementation was manually written using SVE intrinsics defined in the ARM C Language Extensions (ACLE) for SVE without relying on any automatic vectorization. Overall, we took a hierarchical parallelization strategy across MPI, OpenMP and SVE (Figure 6).

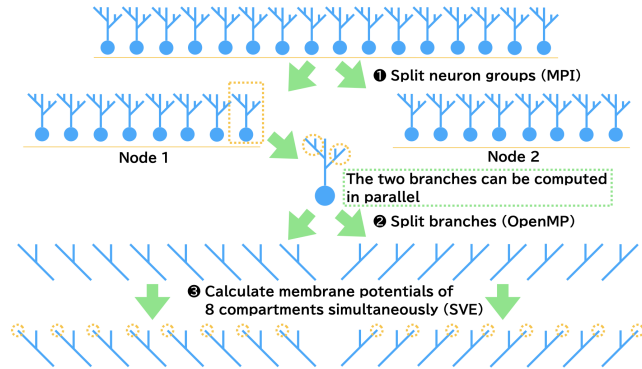


Figure 6: Fully hierarchical parallelization across MPI, OpenMP and SVE for calculation of (d) membrane potential by the following three steps: (1) Neurons in a network are grouped into multiple subgroups on different compute nodes by using MPI. (2) Dendritic branches of a neuron are split and computed in parallel based on previous methods [20, 44] by using OpenMP. (3) During (2), branches for a set of 8 neurons are computed simultaneously by using SVE.

Because the target network was split and distributed across multiple compute nodes, we must exchange spikes over all compute nodes. For each node, spikes were represented by a binary array of size n , where n is the number of neurons assigned on the node. When neuron i emitted a spike, the i th element of the array was set to 1. For the spike propagation, the arrays of spikes on all compute nodes were shared via MPI_Allgather. To compress the amount of transferred data, we packed 8 neurons' spiking activity (i.e., 0 or 1) into an uint8 variable, and broadcast the uint8 arrays.

2.3.1 Performance Measurement. All measurements were carried out on Fugaku in a normal mode at 2.0 GHz of the processor frequency, whereas some compute nodes called assistant nodes ran at the fixed frequency of 2.2 GHz regardless of user-specified settings. The kernel code was compiled with Fujitsu C/C++ Compiler 4.11.2 tcscs-1.2.40 and linked to Fujitsu MPI Library 4.0.0.

To measure FLOPS, we used the Fujitsu “fipp” profiler that measures and outputs statistical information of the entire program through sampling analysis with low overheads [15]. We also used a manual timer instrumentation to measure execution time and a breakdown of the computational time.

2.4 Simulated Network Models

We tested two network models for different purposes: a balanced random network model for benchmark tests and a mouse whole cortex model for demonstrating a biologically realistic simulation.

2.4.1 Balanced Random Network Model. To examine the performance of the simulator, we conducted simulations of a balanced network model with random connections [7], which is a standard benchmark model used in various neuron simulators [6].

A balanced network model consists of N excitatory neurons and M inhibitory neurons, where the ratio $N : M$ is fixed to 4:1, and each neuron receives synaptic inputs from K other neurons chosen uniformly at random. Thus, the network is a random and sparse network when K is small. The model is expected to show synchronized activity across neurons (Figure 7).

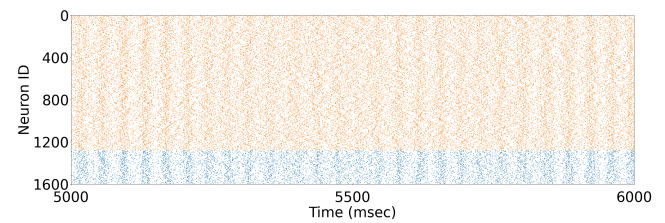


Figure 7: A typical simulation result of the balanced random network model. Dots represent time when a neuron emits a spike. Among the 1,600 neurons, the first 1,280 are excitatory (orange), whereas the rest are inhibitory (blue). Vertical bands of spikes occasionally appear representing synchronized activity of neurons around 30 Hz. Spike data from the initial transient period (first 5 sec) were discarded.

To examine weak scaling, we increased both the numbers of neurons and compute nodes simultaneously while retaining the number of neurons per node constant. We assigned 160 neurons ($N = 128$ and $M = 32$) per node. The network activity for 1 sec of virtual time was simulated. The value of K (the number of synaptic inputs per postsynaptic neuron) was set at 40 throughout the examination. To this end, the largest network on 152,064 compute nodes contained about 24 million neurons and 960 million synapses. On the other hand, to examine strong scaling, the largest network was simulated by using smaller numbers of compute nodes from 152,064 down to 297. Smaller numbers of compute nodes than 297 were unable to simulate the network due to the insufficient amount of memory.

Parameters were fixed throughout simulations as follows: morphology and cell parameter files were Scnn1a_473845048_m.swc and 472363762_fit.json for excitatory neurons, and Pvalb_4705221_02_m.swc and 472912177_fit.json for inhibitory neurons, respectively. Both were taken from “Tutorial: Multi-Population Recurrent Network (with BioNet)” of BMTK. Synaptic connections were made from a presynaptic neuron to the soma of a postsynaptic neuron. The synaptic weights for excitatory and inhibitory connections, decay time constant, and transmission delay were set rather arbitrarily at $1e-7$ and $4e-7$, 5 msec, and 1 msec, respectively.

2.4.2 A Mouse Whole Cortex Model. In a more realistic scenario, we conducted a large-scale network model of a mouse whole cortex (Figure 1). The model consisted of about 9 million biophysical neurons and 26 billion synapses. The neurons were split into 86 groups corresponding to the 86 regions (43 isocortical areas $\times$ 2 hemispheres) in the whole cortex based on the Allen Mouse Brain Common Coordinate Framework [45]. In each region, neurons were arranged in a 3D space as a vertical columnar structure with 5 (motor cortices) or 6 (sensory cortices) vertical layers. Forty-eight types of neurons were included with different morphologies and physiological parameters. Synaptic connections were estimated based on mesoscopic mouse connectome data [31]. Synaptic parameters were estimated based on a large-scale synaptic physiology dataset [8]. Neurons were assumed to receive synaptic inputs only at the soma (perisomatic models) rather than both at the soma and dendrites (all-active models).

3 Results

3.1 Scaling Properties for the Balanced Random Network Model

First, we examined the scaling performance, both weak and strong, on the balanced random network model (Figures 8 and 9).

On weak scaling (Figure 8), computational times for calculations of (b) ion channels, (c) calcium concentration and (d) membrane potentials were ideally constant up to 152,064 nodes. On the other hand, calculation of (a) synaptic currents and spike propagation across compute nodes increased linearly, because the number of communication nodes and the message size increased. The peak performance was 7.13 petaflops when 152,064 compute nodes were used.

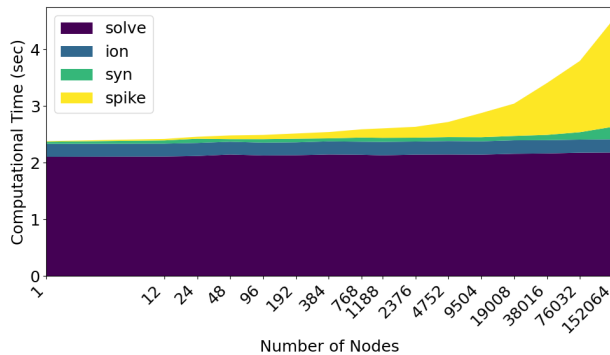


Figure 8: Weak scaling property of the simulation kernel for the balanced random network model for 1 sec of virtual time with a breakdown of the computational time into solving a linear equation for membrane potentials and calcium concentration (solve, purple), calculation of ionic currents (ion, dark blue), calculation of synaptic currents (syn, green) and spike propagation across compute nodes (spike, yellow).

On strong scaling (Figure 9), computational times for calculations of (b) ion channels (c) calcium concentration and (d) membrane potentials decreased ideally as more compute nodes participated

up to 152,064 nodes. On the other hand, the times for calculation of (a) synaptic currents and spike propagation increased gradually. The total computational time decreased upto 152,064 nodes.

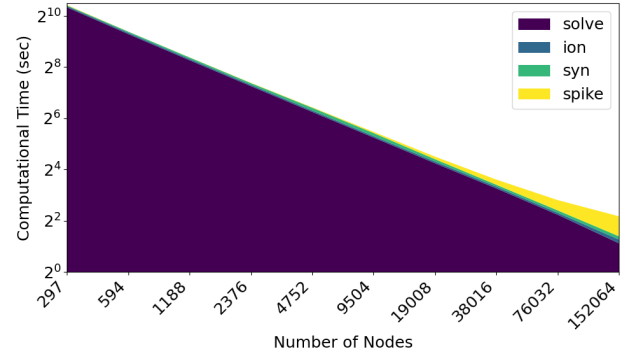


Figure 9: Strong scaling property of the simulation kernel for the balanced random network model with a breakdown of the computational time. Conventions as in Figure 8.

3.2 Simulation of a Mouse Whole Cortex Model

In a more realistic scenario, we conducted simulation of a mouse whole cortex model using 145,728 compute nodes of Fugaku. We fed a constant current to all neurons so that they emit spikes spontaneously at a low firing rate. Although most neurons emitted spikes rather uniformly at random or became silent in response to the tonic current injection, some neuron groups, in particular inhibitory neurons in layer 2/3 of areas 21 and 22 (left and right Frontal Pole), exhibited synchronized activity at around 10 Hz (Figure 10).

Because these neuron groups were distributed in both hemispheres, we wondered if they emitted spikes synchronously through inter-hemispheric interactions. We calculated the population activity of the neurons in each hemisphere, and then calculated the cross-correlation between them (Figure 11). We observed a clear oscillation in the cross-correlation at around 10 Hz, suggesting the inter-hemispheric synchronization.

Finally, we examined the strong scaling for the whole cortex model (Figure 12). The total computational time spent for a simulation for 1 sec of virtual time was at most 32 sec of wall clock time, and the peak performance was estimated approximately 7.35 petaflops when the full-scale 145,728 compute nodes were used.

4 Discussion

The present study introduces software Neulite for microscopic brain simulation. By following the standardized data and data formats provided by the Allen Institute, the simulation kernel became simple and portable, while equipping a fully functional front-end interface. Owing to the portability of the kernel, optimization to Fugaku became rather straightforward. We were able to take a hierarchical parallelization approach across MPI, OpenMP and SVE naturally by correlating them with compute nodes, CMGs and individual cores. The kernel was able to run on Fugaku up to all of the 145,728 compute nodes to simulate a mouse whole cortex model

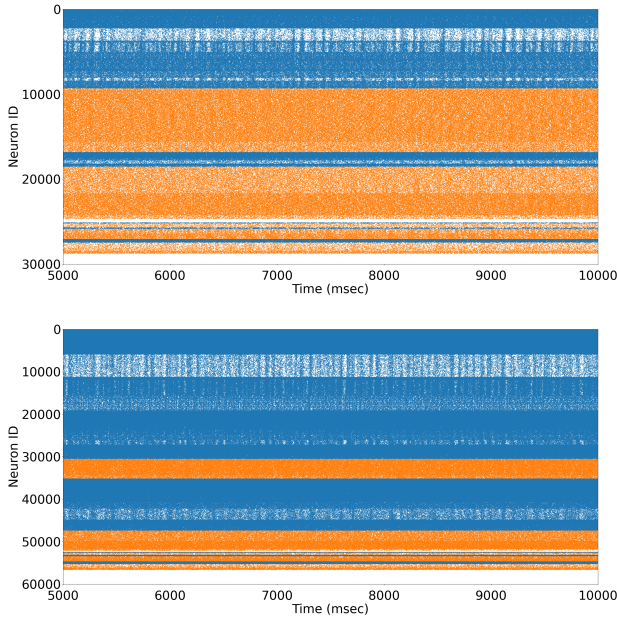


Figure 10: Raster plots of the neurons in areas 21 and 22 (left and right Frontal Pole) showing spikes for the latter 5 sec of the 10 sec simulation (top and bottom panels). Spike data from the initial transient period (first 5 sec) were discarded. Correspondences of neuron IDs and layers (L1, L2/3, L5 and L6) / types (excitatory, exc or inhibitory, inh) are as follows: Left hemisphere: 0–721 (L1inh), 722–9302 (L2/3inh), 9303–16839 (L2/3exc), 16840–18526 (L5inh), 18527–25119 (L5exc), 25120–25190, 25682–25861, 27090–27473 (L6inh), 25191–25681, 25862–27089, 27474–28759 (L6exc); Right hemisphere: 0–700 (L1inh), 701–30488 (L2/3inh), 30489–35221 (L2/3exc), 35222–47363 (L5inh), 47364–52465 (L5exc), 52466–52554, 53167–53382, 54725–55234 (L6inh), 52554–53166, 53383–54724, 55235–56624 (L6exc). Conventions as in Figure 7.

comprising 9 million neurons and 26 billion synapses. This is the largest biophysical model of the mouse cortex to date. Moreover, some neuron groups exhibited synchronized oscillatory activities at around 10 Hz, which can be shown only by whole cortex models.

4.1 Development of the bionet_lite Module

We have developed the bionet_lite module, which is a drop-in replacement of the bionet module that uses the NEURON simulator as the back-end system. The modules' internal structures are completely different, whereas almost the same modeling syntaxes are retained. Our first goal in developing this module was simulator interoperability. Previous neuron simulators had their own application programming interfaces (APIs) so that users had to learn them separately. In our approach, NEURON and Neulite are interchangeable under BMTK, which lowers the learning curve for users to cross-validate simulation results. Our second goal was to

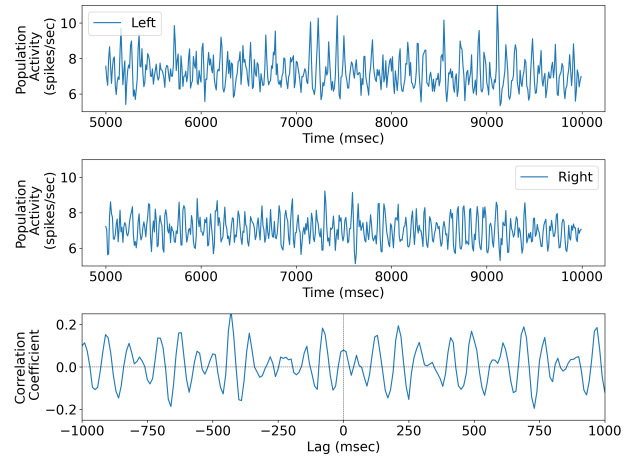


Figure 11: Inter-hemispheric correlation between inhibitory neurons in layer 2/3 of areas 21 and 22. Top and middle: Population activity of the neurons in the left and right hemisphere, respectively, calculated with a time bin of 10 msec. Bottom: Normalized cross-correlation between them.

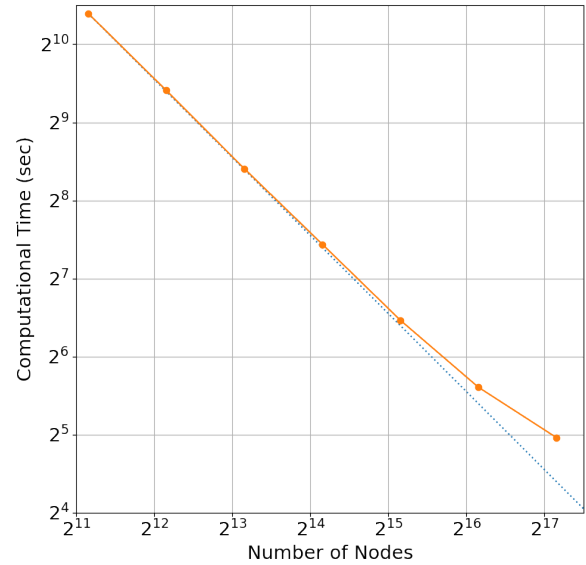


Figure 12: Strong scaling property of the mouse whole cortex model. The numbers of nodes are 2,277, 4,554, 9,108, 18,216, 36,432, 72,864 and 145,728. The dotted line represents the ideal scaling estimated from the data on 2,277 nodes. Both axes are shown in log scale.

encourage the use of standardized data. By following the standardized data in CTDB, simulator development became simple. Thus, the bionet_lite module achieves simulator interoperability and harnesses data standardization.

4.2 Scaling Performance of the Simulation Kernel

On the balanced random network model, we achieved a peak performance of 7.31 petaflops, when the full-scale 152,064 compute nodes participated to simulate 24 million neurons and 960 million synapses. As far as we know, this is the first report of a microscopic brain simulation in petaflops. Furthermore, we still have room to improve the performance significantly. Both weak and strong scaling clearly indicate that communication across compute nodes for spike propagation plays a major role in performance improvement (Figures 8 and 9).

Current state-of-the-art simulators use different algorithms for spike propagation. In the NEURON simulator[9] and Arbor simulator [1], IDs of neurons that emitted spikes are shared across all compute nodes. First, `MPI_Allgather` is called once to let all compute nodes know how many IDs are sent from each compute node. This procedure is necessary to determine the size of the buffer of the IDs on each node. Then, `MPI_Allgatherv` is called to share the IDs across all compute nodes. This method is simple yet efficient when neurons in a network do not emit spikes at high frequency such as neurons in the cerebral cortex. The NEST simulator [18] uses `MPI_Alltoall` [23]. In this approach, neurons are assumed to receive a relatively small number of synapses from other neurons. Because the number of synapses per postsynaptic neuron is independent of the network size, the total amount of data exchanged across compute nodes does not change, even if the network size (i.e., the number of compute nodes participating) increases, resulting in ideal weak scaling. We tested both algorithms in our kernel and observed that the former method did not work well for weak scaling, whereas the latter used much computational time. In particular, `MPI` collective functions on Fugaku were not able to be called asynchronously with a large number of nodes, which were necessary to hide the communication time behind the calculation time. Eventually, we came up to the algorithm that sends the packed binary arrays of spikes. Further study will be necessary to address this issue.

Another potential improvement is to elaborate the network models themselves. In this study, neurons were assumed to have ion channels only at the soma, whereas biological neurons have them everywhere on the membrane surface. These models are called perisomatic and all-active models, respectively. All-active models consume much more computational resources, because all compartments—both somatic and dendritic—have ion channels. Thus, when neuron models are elaborated from perisomatic to all-active, calculation of the ion channels will occupy a large portion of the computational time. Then, we could hide the spike propagation by simultaneous execution of the calculation and communication [23]. Thus, in a more elaborate cortical network model, the overhead of the communication could be negligible or hidden behind the calculation.

4.3 Comparison with Other Neuron Simulators

For biophysical neuron simulation, several alternative simulators exist. The NEURON simulator is currently the de-facto standard software in the field of computational neuroscience [9], while Arbor

is a relatively new software [1]. Both aim to be a complete and self-consistent simulation environment. Specifically, they provide rich features from modeling to visualization, and have their own Python APIs.

Neulite takes the opposite approach. It relies on external standardized databases (CTDB) and the modeling interface (BMTK) completely, thereby focusing on only the numerical calculation. This approach made the simulation kernel small (<2,600 lines of code in the reference implementation) and portable with no dependency on external libraries. Although the kernel was optimized to Fugaku to harness SVEs in this study, the hierarchical parallelization itself in Figure 6 is a universal approach. For example, we could incorporate GPUs or FPGAs instead of SVEs. On portability, the development of Neulite follows UNIX Philosophy exactly [16].

Design choices of the simulator omitted some features found in existing platforms. This decision was made intentionally. The first principle of Neulite kernel is simplicity. To keep the kernel simple, we completely relied on BMTK, and put most functionality to the front-end side. This decision made the kernel portable with no dependency on external libraries, and easy to port to unique supercomputers such as Fugaku. We do not think that the usability deteriorates by this decision, because the modeling and analyses are made in BMTK and other helper apps. Moreover, we can extend the kernel by writing a short code in C. We believe that it is as easy or even easier than writing it in the APIs of existing simulators, owing to the simple and organized data structure.

The core modeling components and numerical methods are well-established in the field, suggesting lack of novelty in algorithmic or systems design. We did this intentionally, again to obtain consistent numerical results with NEURON. In particular, we followed exactly the NEURON-style interpretation of SWC files, which greatly affects the consistency. Thus, algorithms and systems design are made conservative at present. Rather, we can use the kernel as a playground to test more aggressive optimization and advanced numerical techniques in the future.

4.4 Limitations

The current model does not include plasticity, because sufficient biological information is not yet available to inform reliable models of plasticity across the whole cortex. Nevertheless, incorporating hypothetical rules of synaptic and neuronal plasticity into the model and testing their effects will be an important future application, potentially useful for the development of energy-efficient, brain-inspired AI.

We did not provide detailed comparison of our simulation results with those obtained by state-of-the-art simulators. We only showed a small example of the comparison between NEURON and Neulite in the inset of the right panel in Figure 1, which demonstrates how well the voltage traces match between the two simulators. Actually, we have checked the dynamics of single neurons carefully while attaching/detaching each of the ion channels, and confirmed that NEURON and Neulite generated almost identical numerical results at the single neuron level. While we believe that a network-level comparison is important, installing software like NEURON or Arbor, making sure that our huge model is simulated with them, and then characterize performance is a big undertaking. It will be

useful to do this as one of the next steps, but is outside the scope of the current paper, as we are not claiming better performance than other packages. To this end, we only show feasibility and scaling of Neulite.

4.5 Path to Larger-Scale Simulations

The present study demonstrates that a mouse cortex, even at a microscopic level, is already implementable on the current supercomputers. Indeed, the half-scale or even smaller systems as small as 2,277 compute nodes of Fugaku are sufficient for mouse cortex simulation (Figure 12). While the mouse cortex model contains 9 million neurons, the monkey and human cortices contain 480 million and 21 billion neurons, respectively [14, 33]. Thus, a whole monkey cortex could be simulated at a microscopic level using the full-scale system of Fugaku. On the other hand, simulation of the whole human cortex at a microscopic level is still a long-term challenge, although several attempts to carry out human-scale brain simulation have already been made at mesoscopic and macroscopic levels [17, 28, 37, 46]. One estimate suggests that a whole human cortex at a microscopic level will be simulated around the year 2044 or later [21].

5 Conclusion

It is now possible to build a digital replica of the biological brain that retains fine structural and functional properties at a microscopic level on a supercomputer. Such a digital replica, or a “digital brain,” can be used as a ground model for more abstract theoretical/mathematical models, as a clinical tool to study symptoms of various neurological disorders that stem from microscopic properties, and as a reference implementation to develop an energy-efficient brain-style AI, because a human brain uses roughly 20 Watts to work [26]. We hope that Neulite will help move the field of neuroscience towards these broader goals.

Acknowledgments

This study was supported by MEXT/JSPS KAKENHI (JP22H05161, JP22H00460, JP22KJ1372, JP24K22335), The Naito Foundation, The Sasakawa Scientific Research Grant from The Japan Science Society, International Information Science Foundation (2025.1.2.212), AMED Brain/Minds 2.0 (JP24wm0625406, JP25wm0625406), the National Institute of Neurological Disorders and Stroke of the National Institutes of Health (R01NS122742, U24NS124001), the National Institute of Biomedical Imaging and Bioengineering of the National Institutes of Health (R01EB029813), the National Institute of Mental Health of the National Institutes of Health (U01MH130907), and the National Science Foundation (2209873). Computational resources of Fugaku were provided through MEXT Program for Promoting Research on the Supercomputer Fugaku (hp220162, hp230206, hp240214, hp250231). The full-node scale simulation on Fugaku was technically supported by Research Organization for Information Science and Technology as the Advanced User-support Program for Project ID: hp240214. We would like to thank Drs. Kenji Doya, Carlos Gutierrez and Sun Zhe. The content is solely the responsibility of the authors and does not necessarily represent the official views of the National Institutes of Health. We thank the Allen Institute founder, Paul G. Allen, for his vision, encouragement, and support.

References

- [1] Nora Abi Akar, Ben Cumming, Vasileios Karakasis, Anne Kusters, Wouter Klijn, Alexander Peyser, and Stuart Yates. 2019. Arbor — A Morphologically-Detailed Neural Network Simulation Library for Contemporary High-Performance Computing Architectures. In *2019 27th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*. IEEE, Pavia, Italy, 274–282.
- [2] Katrin Amunts, Christoph Ebner, Jeff Muller, Martin Telefont, Alois Knoll, and Thomas Lippert. 2016. The Human Brain Project: Creating a European Research Infrastructure to Decode the Human Brain. *Neuron* 92, 3 (2016), 574–581.
- [3] Anton Arkhipov, Nuno da Costa, Saskia de Vries, Trygve Bakken, Corbett Bennett, Amy Bernard, Jim Berg, Michae Buice, Forrest Collman, Tanya Daigle, et al. 2025. Integrating multimodal data to understand cortical circuit architecture and function. *Nature Neuroscience* 28, 4 (2025), 717–730.
- [4] Giorgio A Ascoli, Duncan E Donohue, and Maryam Halavi. 2007. NeuroMorpho: A Central Resource for Neuronal Morphologies. *Journal of Neuroscience* 27, 35 (2007), 9247–9251.
- [5] Yazan N. Billeh, Binghuang Cai, Sergey L. Gratiy, Kael Dai, Ramakrishnan Iyer, Nathan W. Gouwens, Reza Abbasi-Asl, Xiaoxuan Jia, Joshua H. Siegle, Shawn R. Olsen, and others. 2020. Systematic Integration of Structural and Functional Data into Multi-scale Models of Mouse Primary Visual Cortex. *Neuron* 106, 3 (6 may 2020), 388–403.e18.
- [6] Romain Brette, Michelle Rudolph, Ted Carnevale, Michael Hines, David Beeman, James M Bower, Markus Diesmann, Abigail Morrison, Philip H Goodman, Frederick C Harris Jr, et al. 2007. Simulation of networks of spiking neurons: a review of tools and strategies. *Journal of Computational Neuroscience* 23, 3 (2007), 349–398.
- [7] Nicolas Brunel. 2000. Dynamics of sparsely connected networks of excitatory and inhibitory spiking neurons. *Journal of Computational Neuroscience* 8, 3 (2000), 183–208.
- [8] Luke Campagnola, Stephanie C. Seeman, Thomas Chartrand, Lisa Kim, Alex Hoggarth, Clare Gamlin, Shinya Ito, Jessica Trinh, Pasha Davoudian, Cristina Radaelli, et al. 2022. Local connectivity and synaptic dynamics in mouse and human neocortex. *Science* 375, 6585 (2022), eabj5861.
- [9] Nicholas T. Carnevale and Michael L. Hines. 2009. *The NEURON Book*. Cambridge University Press, Cambridge, UK. <https://www.amazon.co.jp/dp/0521115639>
- [10] MiCRONS Consortium. 2025. Functional connectomics spanning multiple areas of mouse visual cortex. *nature* 640 (2025), 435–447.
- [11] Kael Dai, Sergey L. Gratiy, Yazan N. Billeh, Richard Xu, Binghuang Cai, Nicholas Cain, Atle E. Rimehaug, Alexander J. Stasik, Gaute T. Einevoll, Mihalis, et al. 2020. Brain Modeling Toolkit: An open source software suite for multiscale modeling of brain circuits. *PLOS Computational Biology* 16, 11 (30 nov 2020), e1008386.
- [12] Kael Dai, Juan Hernando, Yazan N. Billeh, Sergey L. Gratiy, Judit Planas, Andrew P. Davison, Salvador Dura-Bernal, Padraig Gleeson, Adrien Devresse, Benjamin K. Dichter, et al. 2020. The SONATA data format for efficient description of large-scale network models. *PLOS Computational Biology* 16, 2 (24 feb 2020), e1007696.
- [13] D.W. Doherty, J. Jung, Dura-Bernal, and W.W. Lytton. 2025. Self-organized and self-sustained ensemble activity patterns in simulation of mouse primary motor cortex. *bioRxiv* 2025.01.13.632866 (2025), 52 pages.
- [14] Aldo Fasolo (Ed.). 2012. *The theory of evolution and its impact*. Springer, Berlin.
- [15] Fujitsu. 2024. *Technical Computing Suite V4.0L20 Development Studio Profiler User's Guide*. Fujitsu. <https://software.fujitsu.com/jp/manual/manualfiles/m240001/j2ul2483/02enz013/j2ul-2483-02enz0.pdf> Accessed: 2025-04-09.
- [16] Mike Gancarz. 2003. *Linux and the Unix Philosophy*. Elsevier, Amsterdam, The Netherlands.
- [17] Daniela Gandolfi, Jonathan Mapelli, Sergio M. G. Solinas, Paul Triebkorn, Egidio D'Angelo, Viktor Jirsa, and Michele Migliore. 2023. Full-scale scaffold model of the human hippocampus CA1 area. *Nature Computational Science* 3 (2023), 264–276.
- [18] Marc-Oliver Gewaltig and Markus Diesmann. 2007. NEST (NEural Simulation Tool). *Scholarpedia* 2, 4 (2007), 1430.
- [19] Michael Hines. 1984. Efficient Computation of Branched Nerve Equations. *International Journal of Bio-Medical Computing* 15 (1984), 69–76.
- [20] Michael L. Hines, Henry Markram, and Felix Schürmann. 2008. Fully implicit parallel simulation of single neurons. *Journal of Computational Neuroscience* 25, 3 (1 dec 2008), 439–448.
- [21] Jun Igarashi. 2024. Future projections for mammalian whole-brain simulations based on technological trends in related fields. *Neuroscience Research* In press (19 nov 2024), 13 pages.
- [22] Allen Institute. 2015. Cell Types. <https://portal.brain-map.org/cell-types> Accessed on March 30, 2025.
- [23] Jakob Jordan, Tammo Ippen, Moritz Helias, Itaru Kitayama, Mitsuhiro Sato, Jun Igarashi, Markus Diesmann, and Susanne Kunkel. 2018. Extremely Scalable Spiking Neuronal Network Simulation Code: From Laptops to Exascale Computers. *Frontiers in Neuroinformatics* 12 (2018), 21 pages.
- [24] Eric R. Kandel, John D. Koester, Sarah H. Mack, and Steven A. Siegelbaum (Eds.). 2021. *Principles of Neural Science* (6th ed.). McGraw-Hill, New York, NY.
- [25] Christof Koch and Idan Segev (Eds.). 2012. *Methods in Neuronal Modeling: From Ions to Networks* (2nd ed.). A Bradford Book, The MIT Press, Cambridge, MA.

- <https://www.amazon.co.jp/dp/0262517132>
- [26] William B Levy and Victoria G. Calvert. 2021. Communication consumes 35 times more energy than computation in the human cortex, but both costs are needed to predict synapse number. *Proceedings of the National Academy of Sciences* 118, 18 (2021), e2008173118. arXiv:<https://www.pnas.org/doi/pdf/10.1073/pnas.2008173118>
- [27] Michael London and Michael Häusser. 2005. Dendritic Computation. *Annual Reviews of Neuroscience* 28 (2005), 503–532.
- [28] Wenlian Lu, Xin Du, Jiexiang Wang, Longbin Zeng, Leijun Ye, Shitong Xiang, Qibao Zheng, Jie Zhang, Ningsheng Xu, Jianfeng Feng, and the DTB Consortium. 2024. Simulation and assimilation of the digital human brain. *Nature Computational Science* 4, 12 (2024), 890–898.
- [29] Henry Markram. 2006. The Blue Brain Project. *Nature Reviews Neuroscience* 7, 2 (1 feb 2006), 153–160.
- [30] Henry Markram, Eilif Müller, Srikanth Ramaswamy, Michael W. Reimann, Marwan Abdellah, Carlos Aguado Sánchez, Anastasia Ailamaki, Lidia Alonso-Nanclares, Nicolas Antille, Selim Arsever, et al. 2015. Reconstruction and Simulation of Neocortical Microcircuitry. *Cell* 163, 2 (8 oct 2015), 456–492.
- [31] Seung Wook Oh, Julie A Harris, Lydia Ng, Brent Winslow, Nicholas Cain, Stefan Mihalas, Quanxin Wang, Chris Lau, Leonard Kuan, Alex M Henry, et al. 2014. A mesoscale connectome of the mouse brain. *Nature* 508, 7495 (2014), 207–214.
- [32] Hideyuki Okano, Erika Sasaki, Tetsuo Yamamori, Atsushi Iriki, Tomomi Shimogori, Yoko Yamaguchi, Kiyoto Kasai, and Atsushi Miyawaki. 2016. Brain/MINDS: A Japanese National Brain Project for Marmoset Neuroscience. *Neuron* 92, 3 (2016), 582–590.
- [33] Steven M Platek, Julian Paul Keenan, and Todd K Shalkeford (Eds.). 2006. *Evolutionary cognitive neuroscience*. The MIT Press, Cambridge, MA.
- [34] Khara M. Ramos, Christine Grady, Henry T. Greely, Winston Chiong, James Eberwine, Nita A. Farahany, L.Syd M. Johnson, Bradley T. Hyman, Steven E. Hyman, Karen S. Rommelfanger, Elba E. Serrano, James D. Churchill, Joshua A. Gordon, and Walter J. Koroshetz. 2019. The NIH BRAIN Initiative: Integrating Neuroethics and Neuroscience. *Neuron* 101, 3 (2019), 394–398.
- [35] Stefan Rotter and Markus Diesmann. 1999. Exact digital simulation of time-invariant linear systems with applications to neuronal modeling. *Biological Cybernetics* 81 (1999), 381–402.
- [36] Oliver Rübél, Andrew Tritt, Ryan Ly, Benjamin K Dichter, Satrajit Ghosh, Lawrence Niu, Pamela Baker, Ivan Soltesz, Lydia Ng, Karel Svoboda, Loren Frank, and Kristofer E Bouchard. 2022. The Neurodata Without Borders ecosystem for neurophysiological data science. *eLife* 11 (oct 2022), e78362.
- [37] Paula Sanz Leon, Stuart A. Knock, M. M. Woodman, Lia Domide, Jochen Mersmann, Anthony R. McIntosh, and Viktor Jirsa. 2013. The Virtual Brain: a simulator of primate brain network dynamics. *Frontiers in Neuroinformatics* 7 (2013), 23 pages.
- [38] Mitsuhsisa Sato, Yuetsu Kodama, Miwako Tsuji, and Tesuya Odajima. 2022. Co-Design and System for the Supercomputer “Fugaku”. *IEEE Micro* 42, 2 (2022), 26–34.
- [39] Alexander Shapson-Coe, Michał Januszewski, Daniel R. Berger, Art Pope, Yuelong Wu, Tim Blakely, Richard L. Schalek, Peter H. Li, Shuohong Wang, Jeremy Maitin-Shepard, and others. 2024. A petavoxel fragment of human cerebral cortex reconstructed at nanoscale resolution. *Science* 384, 6696 (2024), eadk4858.
- [40] Rajagopal Srinivasan and Hillel J. Chiel. 1993. Fast Calculation of Synaptic Conductances. *Neural Computation* 5 (1993), 200–204.
- [41] Nicholas A. Steinmetz, Cagatay Aydin, Anna Lebedeva, Michael Okun, Marius Pachitariu, Marius Bauza, Maxime Beau, Jai Bhagat, Claudia Böhm, Martijn Broux, Susu Chen, et al. 2021. Neuropixels 2.0: A miniaturized high-density probe for stable, long-term brain recordings. *Science* 372, 6539 (2021), eabf4588. arXiv:<https://www.science.org/doi/pdf/10.1126/science.abf4588>
- [42] David B. Stockton and Fidel Santamaria. 2017. Integrating the Allen Brain Institute Cell Types Database into Automated Neuroscience Workflow. *Neuroinformatics* 15, 4 (1 oct 2017), 333–342.
- [43] Nicholas L. Turner, Thomas Macrina, J. Alexander Bae, Runzhe Yang, Alyssa M. Wilson, Casey Schneider-Mizell, Kisuk Lee, Ran Lu, Jingpeng Wu, Agnes L. Bodor, et al. 2022. Reconstruction of neocortex: Organelles, compartments, cells, circuits, and activity. *Cell* 185, 6 (17 mar 2022), 1082–1100.e24.
- [44] Dharma Teja Vooturi, Kishore Kothapalli, and Upinder Singh Bhalla. 2017. Parallelizing Hines Matrix Solver in Neuron Simulations on GPU. In *2017 IEEE 24th International Conference on High Performance Computing (HiPC)*. IEEE, Jaipur, India, 388–397. <https://api.semanticscholar.org/CorpusID:38364548>
- [45] Quanxin Wang, Song-Lin Ding, Yang Li, Josh Royall, David Feng, Phil Lesnar, Nile Graddis, Maitham Naeemi, Benjamin Facer, Anh Ho, et al. 2020. The Allen Mouse Brain Common Coordinate Framework: A 3D Reference Atlas. *Cell* 181, 4 (2020), 936–953.e20.
- [46] Hiroshi Yamaura, Jun Igarashi, and Tadashi Yamazaki. 2020. Simulation of a human-scale cerebellar network model on the K computer. *Frontiers in Neuroinformatics* 14 (2020), 16.

Appendix: Artifact Description/Artifact Evaluation

Artifact Description (AD)

A Overview of Contributions and Artifacts

A.1 Paper's Main Contributions

- C₁ We developed a light-weight biophysical neuron simulator optimized for the supercomputer Fugaku.
- C₂ We measured the scaling performance of the simulator in a balanced random network model, and demonstrated the strong scaling up to 76,032 compute nodes with 1.98 petaflops.
- C₃ We simulated a mouse whole cortex model composed of 9 million biophysical neurons and 26 billion synapses.
- C₄ We analyzed the spike data of the mouse whole cortex model and observed a clear oscillation in the cross-correlation at around 10 Hz, suggesting the inter-hemispheric synchronization.
- C₅ We also fixed a cache thrashing issue in our simulator, and reevaluated the strong scaling in the mouse whole cortex model.

A.2 Computational Artifacts

A₁ <https://doi.org/10.5281/zenodo.15867105>

A₂ <https://doi.org/10.5281/zenodo.16731896>

Artifact ID	Contributions Supported	Related Paper Elements
A ₁	C ₁ , C ₂ , C ₃ , C ₅	Figures 8-10, 12
A ₂	C ₃ , C ₄	Figures 10, 11

B Artifact Identification

B.1 Computational Artifact A₁

A₁ contains the codes and data related to the balanced random network model and the mouse whole cortex model. For the former model, this artifact includes all the codes and data. For the latter model, because the model is proprietary data, this artifact includes only a set of the actual output logs obtained during the simulations.

Relation To Contributions

A₁ provides all the code and data for the balanced random network models for C₁ and C₂. A₁ also provides a set of the actual output logs obtained during the simulations for C₃ and C₅.

Expected Results

This artifact conducts the simulations of the balanced random network model using a hierarchical parallelization strategy across MPI, OpenMP and SIMD parallelism. This artifact then analyzes the spike data of neurons and logs obtained during the simulations, and generates figures of the scaling properties. This artifact also analyzes logs obtained during the simulations of the mouse whole cortex model, and generates a figure of the strong scaling property.

Expected Reproduction Time (in Minutes)

Preparation. The artifact provides all archived files for the simulations of the balanced random network model under 25 different settings. Parallel extraction of all archived files takes approximately 10 minutes.

Compilation. It takes approximately 30 seconds to compile all source codes, build the executable file, and create job scripts.

Execution. Executing all simulations takes approximately 34 hours, excluding job queueing time, to measure scaling property.

Analysis. The entire process, including the analysis of computational time with its breakdown and the generation of figures, takes a few minutes. Measuring FLOPS via “fippox” takes approximately 2 hours in the case of 152,064 nodes.

Artifact Setup (incl. Inputs)

Hardware. The artifact was optimized for Fugaku which utilizes Tofu interconnect D and supports 512 bit-wide vector operations by Scalable Vector Extension (SVE) intrinsics. Fugaku consists of 158,976 compute nodes. Each compute node equips 32GB HBM2 memory and an ARM-based Fujitsu A64FX processor (Armv8.2-A SVE), which is split into 4 compute management groups (CMGs) with 12 compute cores. Each core has 32 registers and 2 SIMD units that execute SVE operations of 512 bits. The inter-node communication is made via Tofu interconnect D with 40GB/sec bandwidth.

Software. Three separate programs (generator, simulator and analyzer) are included. A Python script generates an instance of balanced random network models using Python 3.10.2, NumPy 2.1.3², and pandas 2.2.3³ on a local computer. The simulator is written in C17 and compiled with Fujitsu C/C++ Compiler 4.11.2 tcsds-1.2.40 and linked to Fujitsu MPI Library 4.0.0⁴. Additional header libraries are included in the artifact or generally available on Fugaku without any settings. Another Python scripts analyze data using Python 3.6.8 script with NumPy 1.25.2, pandas 2.1.2, and matplotlib 3.3.4⁵ on Fugaku. We also used the Fujitsu “fipp” profiler⁶ to measure FLOPS.

Datasets / Inputs. Four types of input files are necessary for a simulation: specifications of the morphologies of neurons in the SWC format, specifications for intracellular parameter files in the CSV format, the definitions of neuron populations and synaptic connections in the CSV format.

For balanced random network models, a Python script “sc25_A1/python/gen.py” generates an instance with the number of neurons per node and that of compute nodes specified by users. To reproduce results on scaling properties, one must generate different instances by using this script repeatedly while varying the parameters. A₁ provides all instances we used for scaling properties.

²<https://numpy.org>

³<https://pandas.pydata.org>

⁴The package URL is not open to public.

⁵<https://matplotlib.org/>

⁶<https://software.fujitsu.com/jp/manual/manualfiles/m240001/j2ul2483/02enz013/j2ul-2483-02enz0.pdf>

For the mouse whole cortex model, the data are excluded in this artifact because they are proprietary as mentioned in Section 2.1. Instead, we included the output logs obtained during the simulations, which are used for further analyses, in this artifact.

Installation and Deployment. The requirements are specified in the sections *Hardware* and *Software*. Other external libraries used in this study, such as the SFMT library⁷, are included in the artifact. Therefore, no separate preparation is required. Datasets of balanced random network models are provided as compressed archive files. These files should be put in “sc25_A1/network/balanced_random_network/” directory and then extracted before submitting jobs. The directory structure in this artifact has been described in “README.md”. See also Artifact Execution for further details.

Artifact Execution

To run a simulation, users prepare a job script for a target dataset which defines options (e.g. environment variables, the number of compute nodes, input data, output path, and actual execution command), submit the job to Fugaku, and wait for its completion. The job script outputs computational time and its breakdown on each process to the standard output of each process (stdout. <execution number>. <proc>), MPI statistical information to the standard error of process 0 (stderr. <execution number>. 0), and spike data (spike. <proc>. dat), where <proc> is the MPI rank. This artifact contains all job scripts we used in this study in “sc25_A1/neulite/original_jobscripts/”, and also contains a shell script “submit.sh” that submits all the jobs required to reproduce the results of the balanced random network model simulations. Each job script runs a simulation twice: one generates spike outputs and logs including computational time information, and the other generates data for profiling in “sc25_A1/neulite/log/<job>/prof/basic_prof” via the “fipp” profiler.

Artifact Analysis (incl. Outputs)

To reproduce the figures of scaling property of balanced random network models (Figures 8 and 9), the artifact provides a bash script (analyze.sh). The script generates figures of both weak and strong scaling property (weak_scaling.png, strong_scaling.png). These figures will be stored in “sc25_A1/result/balanced_random_network” directory.

Additionally, the artifact includes a subset of the actual logs from the simulation of the mouse whole cortex model that we conducted. By reading these logs, the bash script outputs a figure strong-scaling-log.png which shows strong scaling property for the mouse whole cortex model (Figure 12). The figure will be located at “sc25_A1/result/mouse_whole_cortex” directory.

To measure FLOPS, the profiling data need to be formatted through “fipp” profiler on the login node of Fugaku. This tool generates human-readable text files that include the information of FLOPS.

B.2 Computational Artifact A_2

Relation To Contributions

A_2 provides the output spike data from the mouse whole cortex model, and their analyzed results. Here, however, the original input data (the description of the neuron populations and synaptic connections across them) cannot be provided for AD/AE, because they are still unpublished. Instead, we provide only the output data and scripts for the analysis.

Expected Results

The artifact reproduces the raster plots and the cross-correlation analysis for the mouse whole cortex model.

Expected Reproduction Time (in Minutes)

Execution. To reproduce the analysis and the results, the total expected elapsed time will be a few seconds on a local computer.

Artifact Setup (incl. Inputs)

Hardware. There are no specific requirements, and the scripts can be run on a local computer.

Software. We wrote Python scripts to create raster plot figures and to analyze the cross correlation, and a Ruby script for extracting population activities. Requirements are Python 3.13.1, numpy 2.2.4, pandas 2.2.3, matplotlib 3.10.1 and Ruby 2.7.0 or higher on a local computer.

Datasets / Inputs. The scripts require the output data (21.dat and 22.dat) that were generated by the mouse whole cortex simulation. As we mentioned above, we only provide the output data. Similarly, the mouse whole cortex data required to fully reproduce Figure 10 are excluded.

Installation and Deployment. The requirements are specified in the sections *Hardware* and *Software*.

Artifact Execution

To reproduce the raster plots (Figure 10), run raster_create_frp.py. It reads the output spike data (21.dat and 22.dat) and generates PNG files (21.dat.png and 22.dat.png). Because the color code for the mouse whole cortex data is in the proprietary data, the result generated by raster_create_frp.py becomes monochrome. To reproduce the cross-correlation analysis (Figure 11), first we extract spike data of L2/3 inhibitory neurons for left and right hemispheres (21.dat and 22.dat, respectively) by running extract_population_activity.rb. It generates l.dat and r.dat. At the same time, the same script reads these files and calculates their normalized population activities (l_norm.dat and r_norm.dat, respectively). Finally, by running calculate_cross_correlation.py, we perform the cross-correlation analysis between l_norm.dat and r_norm.dat and generates the result in a PNG file (cross_correlation_plot.png). All files are read and generated in the working directory.

⁷<https://www.math.sci.hiroshima-u.ac.jp/m-mat/MT/SFMT/index.html>

Artifact Analysis (incl. Outputs)

Three figures (21.dat.png, 22.dat.png, and cross_correlation_plot.png) are created in the working directory. Sample files are included in “output_sample” directory.

Artifact Evaluation (AE)

C.1 Computational Artifact A_1

Artifact Setup (incl. Inputs)

After decompressing “sc25_A1.zip” on Fugaku, move the all compressed data archives of the balanced random network model into “sc25_A1/network/balanced_random_network/” directory. Then, decompress them.

```
$ unzip sc25_A1.zip
# Download and move all .tar.xz to
  sc25_A1/network/balanced_random_network/
$ mv *.tar.xz sc25_A1/network/balanced_random_network/
$ cd sc25_A1/network/balanced_random_network/
# Using multiprocessing is recommended
$ ls *.tar.xz | xargs -P25 -I{} tar -xJf {}
```

Back to sc25_A1, and run “build.sh” to builds executables in the “sc25_A1/neulite/” directory.

```
$ cd path/to/sc25_A1
$ bash build.sh
```

To setup job scripts, run “create_jobscript.sh” and specify an additional file system which is referred to as volume, a group name of user, and a resource group name for access to the full-scale nodes of Fugaku.

```
$ cd path/to/sc25_A1
$ bash create_jobscript.sh
Current working directory: /vol000n/data/hp200xyz/u10jkl/sc25_A1
Volume name: /vol000n
Group name: hp200xyz
Full-scale resource group [optional]: huge_abc

-----
Please review the entered information:

Volume name:          /vol000n
Group name:           hp200xyz
Full-scale rscgrp:     huge_abc
-----
Is this correct? [Y/n]: Y
Proceeding...
Job scripts successfully created.

$
```

This script creates all job scripts in “sc25_A1/neulite/jobscripts/” by modifying our job scripts in “sc25_A1/neulite/original_jobscripts/”. You can obtain both names from the physical path of your data area (/<volumename>/data/<groupname>/).

The full-scale resource group will be provided during a preliminary examination.

Artifact Execution

By default, running “submit.sh” submits all jobs for each dataset required to reproduce the results of the balanced random network model simulations.

```
$ cd path/to/sc25_A1
$ bash submit.sh
```

We should note that jobs requiring 19,008 or more nodes run only during designated periods, which are held monthly. For jobs requiring 76,032 or 152,064 nodes, a preliminary examination is required.

Artifact Analysis (incl. Outputs)

Log files are saved in “sc25_A1/neulite/log/<job>/” directory. The directory contains 3 subdirectories: “output” (standard output and error for each MPI processes), “spike” (spike data), and “prof” (raw profiling data).

To reproduce the figures of scaling property of balanced random network models, run “analyze.sh”.

```
$ cd path/to/sc25_A1
$ bash analyze.sh -f
```

This script analyzes all logs in “sc25_A1/neulite/log/” at once, outputs breakdowns and figures of both weak and strong scaling property to “sc25_A1/result/balanced_random_network/” directory.

To measure FLOPS, navigate to the directory containing “basic_prof” directory, and run “fipp” profiler to analyze data.

```
$ cd path/to/sc25_A1/neulite/log/<target_job>/prof
$ fipp -A -Icpupa -d basic_prof -o summary.txt
```

This command analyze the raw profiling data and generates a file named “summary.txt” that shows performance information including FLOPS metrics.

C.2 Computational Artifact A_2

Artifact Setup (incl. Inputs)

By decompressing the archive sc25_A2.zip on a local computer, a directory (sc25_A2) is created with necessary components in the working directory. It includes the scripts for reproducing the figures and the output data (21.dat and 22.dat) that were generated by the mouse whole cortex simulation.

```
$ unzip sc25_A2.zip
$ cd sc25_A2/
```

Artifact Execution

To reproduce the raster plots (Figure 10), run raster_create_frp.py to generate PNG files (21.dat.png and 22.dat.png).

```
$ python raster_create_frp.py
```

To reproduce the cross-correlation analysis (Figure 11), run extract_population_activity.rb to extract spike data. At the same time, the same script calculates their normalized population activities. Finally, by running calculate_cross_correlation.py, we can perform the cross-correlation and generate the result in a PNG file (cross_correlation_plot.png).

```
$ ./extract_population_activity.rb
$ python calculate_cross_correlation.py
```

Each script needs the output data (21.dat and 22.dat) in the working directory.

Artifact Analysis (incl. Outputs)

Three figures (21.dat.png, 22.dat.png, and cross_correlation_plot.png) are created in the working directory.